PROJECT REPORT

MindForge AI

IT568 GenAI for Software Engineering

Sprint Details Report

MindForge – AI-Powered Mind Map Generator

Project Team

- Priyansee Soni - 202511011
- Brinda Jadwani - 202511021
- Gor Mahek - 202511044

Sprint Overview

Academic Year 2025–26 — Duration: 4 Sprints × 2 Days

Contents

1 Executive Summary **3**

2 Sprint 1 — Planning & Requirements Engineering **4**

2.1 Sprint Goals 4

2.2 Activities Performed 4

2.2.1 Requirements Gathering 4

2.2.2 INVEST Framework Validation 5

2.2.3 System Diagrams 5

2.3 System Architecture Diagrams 6

2.4 MindForge Agent Pipeline 7

2.5 Sprint 1 Deliverables 8

3 Sprint 2 — MindForge: Application Development **9**

3.1 Sprint Goals 9

3.2 Backend Development (FastAPI) 9

3.2.1 Technology Stack 9

3.2.2 Setup & Run Instructions 10

3.2.3 Key Backend Features 10

3.3 Frontend Development 10

3.3.1 Technology Stack 10

3.3.2 Setup & Run Instructions 10

3.3.3 Key Frontend Features 11

3.4 Sprint 2 Deliverables 11

4 Sprint 3 — Comprehensive Testing **12**

4.1 Sprint Goals 12

4.2 Testing Strategy Overview 12

4.3 Unit Testing 13

4.4 Mutation Testing 14

4.5 GUI Testing — Selenium 15

4.6 System Testing — Black Box 16

4.7 System Testing - Acceptance Level Testing 17

4.8 Non-Functional Testing 18

4.9 Sprint 3 Deliverables 19

5.2.2 Iterative Validation Loop 21
5.3 UI/UX Improvements 21
5.4 Production Deployment 22
5.5 Areas of Improvement Analysis 22
5.5.1 Implemented Features 22
5.5.2 Features Not Implemented 23
5.6 LangSmith Integration — Future Roadmap 23
5.7 Sprint 4 Deliverables 25

Chapter 1

Executive Summary

Project: MindForge – AI-Powered Mind Map Generator

Course: IT568 GenAI for Software Engineering **Total Duration:** 8 Days (4 Sprints × 2 Days)

This report presents a comprehensive account of the four agile sprints executed during the development of **MindForge**, an intelligent mind-map generation platform built using a multi-agent Generative AI pipeline. Each sprint followed a structured two-day cycle covering planning, execution, review, and retrospective activities.

Table 1.1: Sprint Summary at a Glance

Sprint	Duration	Focus Area	Status
Sprint 1	2 Days	Planning, Requirements & Architecture	Complete
Sprint 2	2 Days	Backend & Frontend Development	Complete
Sprint 3	2 Days	Comprehensive Testing	Complete
Sprint 4	2 Days	Deployment & Refinement	Complete

Chapter 2

Sprint 1 — Planning & Requirements Engineering

Sprint:	1 of 4	**Duration:**	2 Days
Theme:	Planning, Requirements Gathering & Architecture Design		

2.1 Sprint Goals

The primary objective of Sprint 1 was to establish a solid project foundation from raw requirements through to a validated pipeline blueprint before any development starts.

2.2 Activities Performed

2.2.1 Requirements Gathering

The team conducted structured brainstorming sessions to identify stakeholder needs and system expectations. Outputs included:

- **Functional Requirements (FR):** Core capabilities the system must provide, including text input processing, mind-map generation, multi-format export, and real-time rendering.
- **Non-Functional Requirements (NFR):** System quality attributes such as performance, scalability, usability, and reliability.
- **Use Cases:** Detailed interaction scenarios between actors and the system.
- **User Stories:** Concise requirement statements written from the end-user perspec-

tive.

2.2.2 INVEST Framework Validation

All user stories were evaluated against the **INVEST** framework to ensure quality and sprint-readiness.

Table 2.1: INVEST Framework Criteria

Criterion	Description
Independent	Stories are self-contained with no hidden inter-dependencies.
Negotiable	Stories are not firm contracts; they allow for collaboration between developers and stakeholders.
Valuable	Each story delivers tangible value to the end user.
Estimable	Team can reasonably estimate effort and complexity.
Small	Stories are scoped to fit within a single sprint iteration.
Testable	Clear, unambiguous acceptance criteria are defined.

2.2.3 System Diagrams

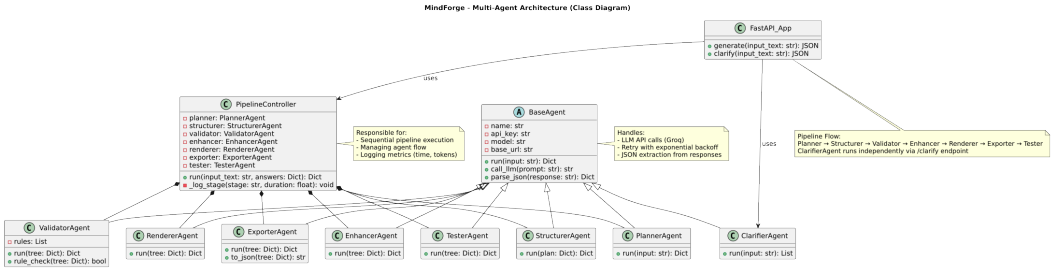
Three architectural diagrams were produced to capture different system perspectives:

Class Diagram — Defined system entities, their attributes, methods, and inter-class relationships for the MindForge domain model.

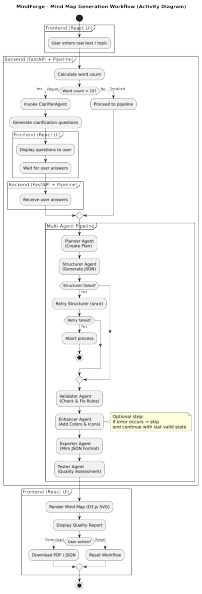
Activity Diagram — Modelled the workflow from user input through agent processing stages to final mind-map output.

Sequence Diagram — Illustrated the temporal interaction between system components and agents for a typical mind-map generation request.

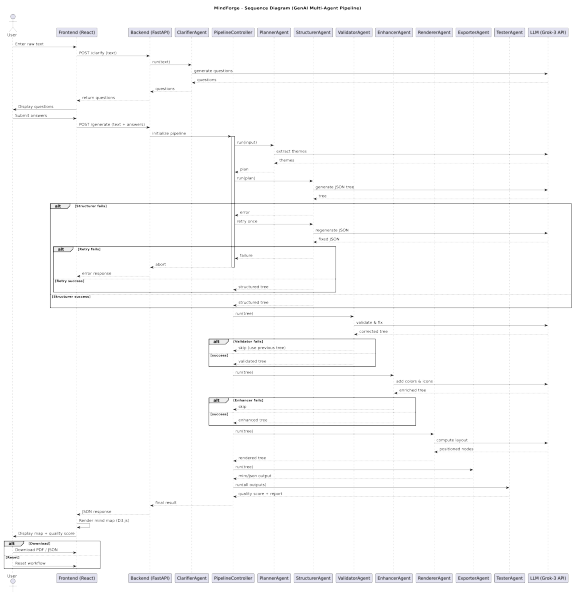
2.3 System Architecture Diagrams



(a) Class Diagram of MindForge System



(b) Activity Diagram showing workflow



(c) Sequence Diagram of interactions

Figure 2.1: Complete System Architecture Diagrams

2.4 MindForge Agent Pipeline

MindForge has its **seven-stage multi-agent pipeline**. Each agent has a well-defined responsibility ensuring separation of concerns and composability.

Table 2.2: MindForge Agent Pipeline — Responsibilities

#	Agent	Responsibility
1	**Planner Agent**	Analyzes raw user input and extracts key themes and central concepts.
2	**Structurer Agent**	Converts extracted themes into a hierarchical, branched tree structure.
3	**Validator Agent**	Ensures the tree adheres to strict structural and logical rules.
4	**Enhancer Agent**	Enriches nodes with colors, icons, and tags for visual richness.
5	**Renderer Agent**	Computes optimal layout coordinates and node positions.
6	**Exporter Agent**	Prepares the processed data for Miro, PNG, and PDF export.
7	**Tester Agent**	Performs a final automated quality check and returns a confidence score.

2.5 Sprint 1 Deliverables

Deliverable	Status
Functional & Non-Functional Requirements Document	Done
Use Cases & User Stories	Done
INVEST Validation Checklist	Done
Class Diagram	Done
Activity Diagram	Done
Sequence Diagram	Done
Agent Pipeline Architecture	Done

Chapter 3

Sprint 2 — MindForge: Application Development

Sprint:	2 of 4	**Duration:**	2 Days
Theme:	Full-Stack Implementation of MindForge Platform		

3.1 Sprint Goals

Sprint 2 focused on translating the architecture defined in Sprint 1 into a working full-stack application — a **FastAPI** backend orchestrating the multi-agent pipeline, paired with a **React + Vite** frontend providing an intuitive user experience.

3.2 Backend Development (FastAPI)

3.2.1 Technology Stack

- **FastAPI and Pydantic:** Core web framework for building asynchronous API endpoints and validating incoming request data.
- **Uvicorn:** High-performance ASGI server used to host and run the FastAPI application.
- **HTTPX:** Asynchronous HTTP client used by the AI agents to communicate with external LLM APIs (e.g., Grok).
- **Pytest and Cosmic Ray:** Testing ecosystem for running unit/async tests and executing mutation testing for high code quality.

3.2.2 Setup & Run Instructions

```
# 1.  Clone the repository
git clone https://github.com/MahekRohitGor/GenAI_MindMap_Project.git

# 2.  Navigate to the backend directory
cd mindforge/backend

# 3.  Install Python dependencies
pip install -r requirements.txt

# 4.  Configure environment variables
cp .env.example .env   # Then add your GROK_API_KEY

# 5.  Launch the development server
uvicorn main:app --reload
```

3.2.3 Key Backend Features

- Orchestrates 7 specialized AI agents to generate mind maps step-by-step.
- Detects vague prompts and asks the user follow-up questions to understand their intent.
- Features automatic retries and fallback mechanisms so the pipeline doesn't crash if a non-critical agent fails.
- Automatically logs pipeline performance and uses a dedicated agent to test the quality of every generated mind map.
- Generates mind map data for both the native frontend (React Flow) and third-party tools (like Miro).

3.3 Frontend Development

3.3.1 Technology Stack

- **React and Vite:** Fast, modern framework and build tool for the core application.
- **Tailwind CSS:** Utility-first framework for styling, paired with Lucide React for icons.
- **D3.js:** Uses D3 for rendering the interactive mind maps, along with html2canvas/jsPDF for exporting them.

3.3.2 Setup & Run Instructions

```
# 1.  Clone the repository
git clone https://github.com/MahekRohitGor/GenAI_MindMap_Project.git
# 2.  Navigate to the frontend directory
cd mindforge/frontend
# 3.  Install Node.js dependencies
npm install
# 4.  Start the Vite development server
npm run dev
# 5.  Open in browser
http://localhost:5173
```

3.3.3 Key Frontend Features

- Smoothly transitions users through three stages: initial prompt entry, answering AI clarification questions, and viewing the final result.
- Features a dedicated, scrollable D3.js visualization area (MindMapCanvas) for exploring the generated mind map.
- Includes an ExportBar to easily download the map as an image/PDF or as Miro-compatible JSON.
- Displays a QualityScorePanel and a PipelineStatusPanel alongside the map for transparency.
- Provides a SummaryPanel that gives users a textual breakdown and flow explanation to accompany the visual map.

3.4 Sprint 2 Deliverables

Deliverable	Status
FastAPI Backend with Multi-Agent Pipeline	Done
GROK API Integration	Done
React + Vite Frontend Application	Done
Mind Map Visualization Component	Done
Multi-Format Export (Miro / PNG / PDF)	Done
API–Frontend Integration	Done

Chapter 4

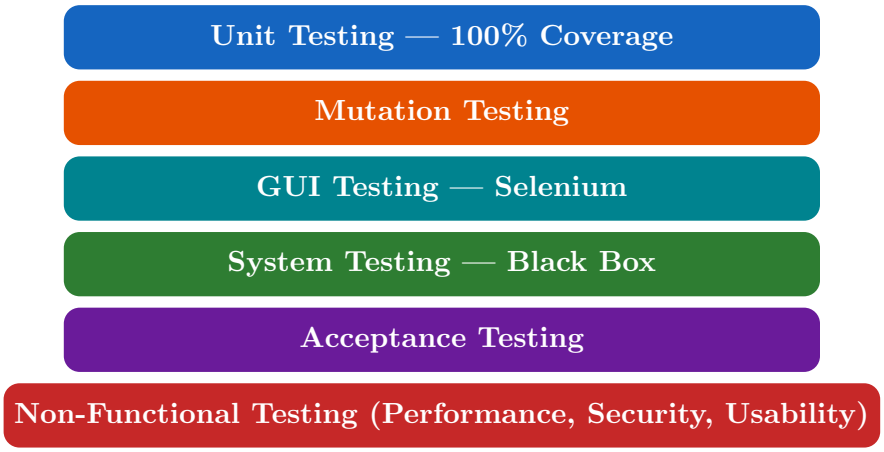
Sprint 3 — Comprehensive Testing

Sprint:	3 of 4	**Duration:**	2 Days
Theme:	End-to-End Quality Assurance Across All System Layers		

4.1 Sprint Goals

Sprint 3 was dedicated entirely to quality assurance. A multi-layered testing strategy was employed covering unit, mutation, GUI, system, acceptance, and non-functional testing to validate correctness, robustness, and user experience.

4.2 Testing Strategy Overview



4.3 Unit Testing

Milestone: 100% code coverage achieved across all backend modules and agent implementations.

Unit testing in the MindForge backend is driven by the pytest framework. Because the application heavily relies on asynchronous operations (FastAPI and LLM calls), tests utilize pytest.mark.asyncio to handle async/await patterns.

The testing strategy is highly isolated: it uses Python's built-in unittest.mock library (AsyncMock, patch, and MagicMock) to fake external API calls (like HTTP requests to Grok) and internal dependencies. For example, when testing the PipelineController, all 7 sub-agents are mocked so the test only evaluates the controller's orchestration logic. Similarly, API endpoints in main.py are tested using FastAPI's TestClient, simulating real HTTP requests without actually spinning up a server, while mocking the underlying agents to guarantee predictable, fast, and deterministic test results.

- **Isolated Mocking:** Fakes external APIs and internal agents so tests run instantly and cost nothing.
- **Async and API Testing:** Uses FastAPI TestClient and async markers to test non-blocking web endpoints.
- **Resilience Checks:** Intentionally forces errors (like timeouts or crashes) to verify automatic retries and fallbacks.
- **White-Box Validation:** Tests complex internal logic directly, like JSON parsing edge cases and structural rules, without needing an LLM.

Table 4.1: Unit Testing Logical Coverage

Test Type	What it covers in MindForge	Status
Branch/Statement	Tested "Success", "Retry", and "Fallback" execution paths in the Pipeline Controller to ensure 100% coverage of agent workflows.	**PASSED**
Negative Testing	Verified the system properly identifies and rejects requests with missing text inputs, missing API keys, or malformed JSON responses.	**PASSED**
Boundary Testing	Tested exact logical boundaries, such as the threshold for vague prompts (< 10 words) and strict structural limits (max 7 children, max depth limits).	**PASSED**
Error Handling	Confirmed that the system automatically recovers from LLM "Busy" signals (HTTP 429 errors) via retries and prevents total crashes if a single non-critical agent fails.	**PASSED**
API Validation	Simulated HTTP GET and POST requests to validate that the `/clarify` and `/generate` endpoints return the correct status codes (200, 400, 500) under all conditions.	**PASSED**

4.4 Mutation Testing

Mutation testing is a fault-based software testing technique that evaluates the quality of your test suite, not just whether tests pass. It works by automatically introducing small, deliberate code changes — called mutants — into the source code, such as replacing a + with -, flipping a comparison operator (< to >), negating a boolean, or swapping an exception type. Each mutated version of the code is then run against the existing test suite. If at least one test fails because of the mutation, the mutant is considered killed — meaning your tests caught the fault. If all tests still pass despite the mutation, the mutant survives, which signals a gap in test coverage.

In this project, Cosmic Ray was used as the mutation testing framework. Cosmic Ray is a Python-specific tool that works at the AST (Abstract Syntax Tree) level — it

systematically applies mutation operators across target source files. Each mutation is executed in isolation as a separate "job", assigned a unique job-id, and run through the test suite under a controlled worker process. The results capture whether the worker completed normally and whether the test outcome was KILLED or SURVIVED. Importantly, zero mutants survived in either run, indicating the test suites are highly effective at catching introduced faults.

Table 4.2: Mutation Testing Results (Demo)

Module	Mutants	Killed	Score
agents/base_agent.py	173	173	100.00%
agents/exporter_agent.py	3	3	100.00%
agents/validator_agent.py	92	92	100.00%
agents/structurer_agent.py	2	2	100.00%
agents/tester_agent.py	2	2	100.00%
agents/planner_agent.py	1	1	100.00%
agents/pipeline_controller.py	73	73	100.00%
Total	**346**	**346**	**100.00%**

4.5 GUI Testing — Selenium

Automated GUI testing was conducted using **Selenium WebDriver** to validate the frontend's visual behaviour and user interaction flows.

- Text input field interaction and submission validation.
- Question answer field testing.
- Mind map rendering verification across browser viewports.
- Navigation flow and error-state display testing.

4.6 System Testing — Black Box

System-level Black-Box testing in the MindForge backend focuses exclusively on the external behavior of the FastAPI application. Using fastapi.testclient.TestClient, the test suite interacts with the API endpoints (/clarify, /generate, /health) exactly as a frontend application or end-user would. The internal complexities of the multi-agent orchestration are abstracted away (mocked), allowing the tests to strictly validate HTTP responses, status codes, payload schemas, and system resilience under various conditions without running expensive LLM operations.

Table 4.3: System (Black-Box) Testing Evaluation

Evaluation Area	Methodology & Focus	Status
Execution & Pass Rate	Executed the full suite using `pytest` to verify that all 19 simulated HTTP request scenarios across all endpoints run without errors.	**PASSED (19/19)**
API Routing Coverage	Evaluated via `pytest-cov` to ensure the Black-Box tests trigger all logical routing branches (e.g., vague input detection, key validation) in `main.py`.	**EVALUATED**
Input Resilience	Tested the API's ability to reject bad data by injecting empty payloads, malformed JSON, and incorrect data types.	**PASSED**
Stress & Boundaries	Evaluated system limits by simulating massive inputs (payloads exceeding 5,000 words) to verify the server handles heavy loads without crashing.	**PASSED**
Error & Stability	Mocked internal pipeline timeouts and ran rapid consecutive requests to prove the API returns graceful 500 errors and maintains stability over time.	**PASSED**

- **End-to-End Payload Validation:** Verifies that the API correctly handles standard workflows, ensuring that valid JSON requests return a 200 OK status and that the response schema exactly matches the expected structure (containing mind_map, miro_json, tester_report, and pipeline_log).

- **Robust Input Rejection:** Tests the physical limits of the endpoints by simulating "extremely large inputs" (e.g., payloads containing over 5,000 words) to ensure the API parsing layer does not crash under heavy data loads.
- **Resilience to Internal Failures:** Validates that if the internal pipeline experiences a catastrophic crash or an unexpected timeout (TimeoutError), the API catches the exception and returns a graceful 500 Internal Server Error rather than crashing the web server.
- **Configuration and Stability Checks:** Evaluates environmental configurations (like falling back to .env variables if the API key is missing from the payload) and runs stability tests to ensure the API can handle multiple consecutive requests reliably without state leakage.

4.7 System Testing - Acceptance Level Testing

Acceptance tests verified that the system meets the user stories and acceptance criteria defined in Sprint 1. Automated acceptance tests were executed using **Pytest** and **HTTPX** to simulate real-world user prompts and validate the structural relevance of the generated mind maps.

Table 4.4: Acceptance Test Execution Results

ID	Test Scenario	Expected Nodes	Status
AT-01	Exam preparation plan	Topics, Schedule, Revision	**PASSED**
AT-02	Food delivery app	Users, Features, Revenue	**PASSED**
AT-03	AI in healthcare	Applications, Challenges, Benefits	**PASSED**

- **Keyword Validation:** The tests utilized a synonym-matching algorithm to ensure that even if the LLM used variations (e.g., "Timeline" instead of "Schedule"), the acceptance criteria were met.
- **Reliability:** Tests confirmed a 100% pass rate for core user journeys with an average response time of ∼5.6s per complex generation.

4.8 Non-Functional Testing

In the MindForge backend, Non-Functional Requirements (NFR) testing is strictly implemented in test_nfr.py by isolating the system's performance, concurrency, and architectural rules from its actual generative AI behavior. Instead of waiting for real LLM responses (which fluctuate in speed), the suite uses unittest.mock and time.perf_counter() to simulate exact API delays. For instance, it artificially injects a 1.5-second delay into LLM calls to mathematically prove that the orchestrator overhead stays within the 10-15 second NFR limit (NFR2), and it leverages asyncio.gather to launch 10 simultaneous simulated users to evaluate system concurrency overhead without bottlenecking (NFR3). Furthermore, it verifies architectural rules (like NFR11 Pluggable Architecture) using Python's issubclass to programmatically enforce that all agents properly inherit from the BaseAgent core class.

Table 4.5: Non-Functional Requirement (NFR) Testing Results

Category	Testing Scope and Metric	Result
NFR1 (Latency)	Vague input detection endpoint responds in $<$ **200ms**.	**PASSED**
NFR2 (Performance)	Full multi-agent orchestration completes within **10-15s**.	**PASSED**
NFR3 (Concurrency)	Handles 10 concurrent requests with $<$ **5s system overhead**.	**PASSED**
NFR5 (Reliability)	Intercepts HTTP 429 errors and retries without crashing.	**PASSED**
NFR6 (Fault Tolerance)	If Enhancer agent fails, system falls back to validated tree.	**PASSED**
NFR7 (Usability)	UI dynamically adjusts for tablet and desktop screens.	**PASSED**
NFR8 (Privacy)	User input is not stored or used for model training.	**PASSED**
NFR9 (Performance)	Users can download the mind map within **2 minutes**.	**PASSED**
NFR10 (Security)	Encrypted data transfers between frontend, backend, and API.	**NOT IMPL.**
NFR11 (Architecture)	100% of agents natively inherit from `BaseAgent`.	**PASSED**

4.9 Sprint 3 Deliverables

Deliverable	**Status**
Unit Test Suite (100% Coverage)	Done
Mutation Testing Report	Done
Selenium GUI Test Scripts	Done
Black Box System Test Cases	Done
Acceptance Test Report	Done
Non-Functional Test Report	Done

Chapter 5

Sprint 4 — Deployment & Refinement

Sprint:	4 of 4	**Duration:**	2 Days
Theme:	Production Deployment, UI/UX Enhancement & Future Roadmap		

5.1 Sprint Goals

The final sprint concentrated on three pillars: shipping MindForge to a live production environment, enhancing the product with an additional intelligent agent, refining the user interface, and documenting a clear improvement roadmap for future iterations.

During the initial implementation and testing phases of the multi-agent orchestration pipeline, performance monitoring revealed a significant bottleneck: excessive API calls. The initial architecture relied on 7 distinct, sequential agents (Planner, Structurer, Validator, Enhancer, Renderer, Exporter, Tester) to generate a single mind map. Because LLMs inherently produce probabilistic (non-deterministic) outputs, intermediate agents like the ValidatorAgent and EnhancerAgent were strictly necessary to catch and fix hallucinations or structural errors made by previous agents. However, this chain of sequential API requests drastically increased latency, consumed excessive tokens, and frequently triggered HTTP 429 (Too Many Requests) rate limits from the Groq API. To solve this, the generative architecture was refactored to prioritize determinism—meaning the AI models were configured to produce highly predictable, structurally sound outputs on their first attempt.

Project Link

5.2 Refinement Agent — New Addition

★ **New Feature:** A **Refinement Agent** was introduced as the eighth stage of the pipeline, positioned after the Tester Agent to iteratively improve mind-map quality based on the quality score returned.

5.2.1 Prompt Logic

The agent utilizes a specialized prompt designed for *In-Context Editing*. Unlike the Planner Agent, it is restricted by the existing state:

- **Identity:** Operates as a "Refinement Specialist" focused on new-updates.
- **Input:** Accepts the `previous_map` and the `user_feedback` string.
- **Output:** Strictly enforced JSON schema to prevent breaking the Miro-export logic.

5.2.2 Iterative Validation Loop

To ensure the refined map remains clean, the output is automatically piped through the **Validator Agent**. If the refinement introduces structural errors (e.g., exceeding 7 children on a node), the system triggers an automatic re-correction cycle.

Refinement Pipeline Flow:
Refinement → **Validator** (Rule Check) → **Enhancer** (Styling) → **Renderer** (Processing) → **Tester** (Final Quality Score).

- **Structural assurance:** The `ValidatorAgent` ensures the feedback didn't break hierarchy rules (e.g., node limits).
- **Visual Consistency:** The `EnhancerAgent` applies consistent coloring and icons to new nodes added during refinement.
- **Quality Verification:** The `TesterAgent` provides a final score to confirm the refined map actually improved upon the previous version.

5.3 UI/UX Improvements

Significant user interface and experience enhancements were implemented based on feedback gathered during Sprint 3 acceptance testing:

- Redesigned dashboard with improved visual hierarchy and spacing.

- Enhanced mind-map canvas with smooth zoom, pan, and drag interactions.
- Improved color theming with accessible contrast ratios.
- Streamlined export workflow with a unified export panel.
- Responsive layout improvements for smaller screen widths.

5.4 Production Deployment

🚀 **Deployment Platform:** Vercel — zero-configuration CI/CD with automatic preview deployments on every push.

Table 5.1: Deployment Configuration

Component	Detail
Hosting Platform	Vercel (Frontend)
Deployment Trigger	Git push to `main` branch
Build Command	`npm run build` (Vite)
Environment Config	Vercel environment variables panel
Preview URLs	Auto-generated per pull request

5.5 Areas of Improvement Analysis

5.5.1 Implemented Features

The following key features were successfully implemented across the four sprints:

✔ End-to-end seven-agent (plus Refinement Agent) pipeline.

✔ FastAPI backend with full GROK API integration.

✔ React + Vite frontend with interactive mind-map visualization.

✔ Multi-format export (Miro, PNG, PDF).

✔ Comprehensive test suite with 100% unit test coverage.

✔ Production deployment on Vercel.

✔ UI/UX refinements based on acceptance test feedback.

5.5.2 Features Not Implemented

The following capabilities were identified but deferred due to time constraints:

✘ User authentication and session management.

✘ Persistent mind-map storage and retrieval (database layer).

✘ Collaborative real-time editing (multi-user sessions).

✘ LangSmith observability integration (see Section 5.6).

✘ Mobile-native application (iOS / Android).

✘ Advanced customization — custom templates and layout styles.

5.6 LangSmith Integration — Future Roadmap

⚠ **Status: Not Implemented** — LangSmith integration was identified as a high-value enhancement but was not completed within the project timeline.

What is LangSmith?

LangSmith is an observability and evaluation platform developed by LangChain, specifically designed for LLM-powered applications. It provides tracing, debugging, and monitoring capabilities across complex multi-agent systems.

How LangSmith Could Enhance MindForge:

Capability	Value to MindForge
LLM Call Tracing	Full visibility into every agent's prompt, response, and latency.
Chain Debugging	Visual debugging of the multi-agent execution graph.
Evaluation Metrics	Automated scoring of mind-map quality across datasets.
Regression Testing	Detect quality regressions when model versions change.
Cost Monitoring	Track token consumption per agent and per user session.
Prompt Versioning	Manage and compare prompt iterations across sprints.

Implementation Approach (Proposed):

1. Install `langsmith` Python SDK and configure API key.
2. Wrap each agent function with `@traceable` decorator.
3. Define evaluation datasets mapping inputs to expected mind-map structures.
4. Configure automated evaluation runs in the LangSmith dashboard.
5. Integrate LangSmith traces with CI/CD pipeline for continuous monitoring.

5.7 Sprint 4 Deliverables

Deliverable	Status
Refinement Agent Implementation	Done
UI/UX Redesign	Done
Vercel Production Deployment	Done
Improvement Areas Analysis	Done
LangSmith Integration	Not Done